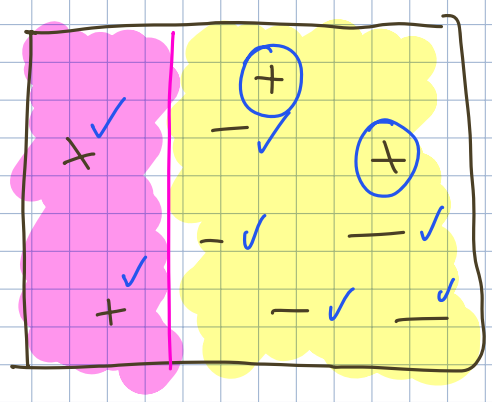
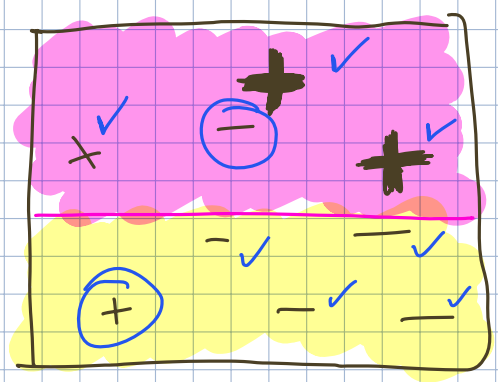
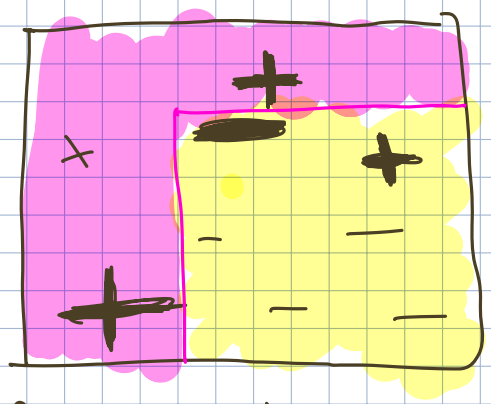
Adaboost: en cada iteración del algoritmo construimos un árbol de clasificación y le damos más peso a las observaciones donde el algoritmo se equivocó en la iteración pasada:







$(x_1, y_1), \dots (x_N, y_N)$

x_i: features (vector p-dimensional)

Total de N observaciones.

Primero entrenamos un árbol $g^{[0]}(x)$ para predecir y.

En esta primera iteración, los pesos de todas las observaciones son idénticos.

Vemos en qué puntos se equivocó $g^{[0]}(x)$, actualizamos pesos y entrenamos $g^{[1]}(x)$ ...

La predicción final:

$$\hat{f}_{ada}(x_i) = \sum_{j=1}^{M} \alpha_j \, g^{[j]}(x_i)$$

M es el número de árboles

predicción para x_i del modelo $g^{[j]}$

pesos dependen del desempeño del modelo j

En la ~~iteración~~ 1:

$$w_i^{[1]} = \frac{1}{N}$$

Para $2, \dots, M$:

construimos el modelo $g^{[m]}(\cdot)$ utilizando los pesos $w^{[m]}$:

$$error^{[m]} = \frac{\sum_{i=1}^{N} w_i^{[m]} \mathbb{1}\{g_i^{[m]}(x_i) \neq y_i\}}{\sum_{i=1}^{N} w_i^{[m]}}$$

$$\alpha^{[m]} = \log\left(\frac{1 - error^{[m]}}{error^{[m]}}\right)$$

$$\tilde{w}_i^{[m+1]} = w_i^{[m]} \exp\left(\alpha^{[m]} \underbrace{\mathbb{1}\{g^{[m]}(x_i) \neq y_i\}}\right)$$

si modelo en iteración m se equivoco $\Rightarrow$ $=1$

$w_i^{[m]} \exp(\alpha^{[m]})$

si modelo acertó:

$w_i^{[m]} \exp(0) = w_i^{[m]}$.

$$w_i^{[m+1]} = \frac{\tilde{w}_i^{[m+1]}}{\sum_{i=1}^{N} \tilde{w}_i^{[m+1]}}$$

Gradient boosting (XGboost):

$(x_1, y_1), \dots, (x_N, y_N)$

Lo primero que hacemos es entrenar un árbol $f^{[1]}(x)$ para predecir y:

			errores:
$y_1 = 0.9$	$f^{[1]}(x_1) = 0.8$		$0.8 - 0.9 = -0.1$
$y_2 = 1.4$	$f^{[1]}(x_2) = 1.5$	$\Rightarrow$	$1.5 - 1.4 = 0.1$
$y_3 = 6.2$	$f^{[1]}(x_3) = 6.0$		$6.0 - 6.2 = -0.2$
$\vdots$	$\vdots$		$\vdots$

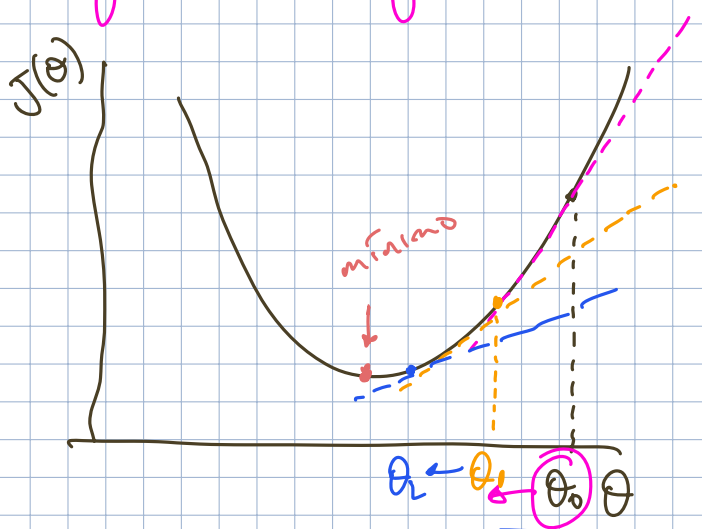
Supongamos que entrenamos un segundo modelo $f^{[2]}$ para predecir los negativos de esos errores:

$$y_1 - f^{[1]}(x_1) = 0.1$$
$$y_2 - f^{[1]}(x_2) = -0.1$$
$$y_3 - f^{[1]}(x_3) = 0.2$$
$$\vdots$$

supongamos que $f^{[2]}$ predice perfectamente estos valores

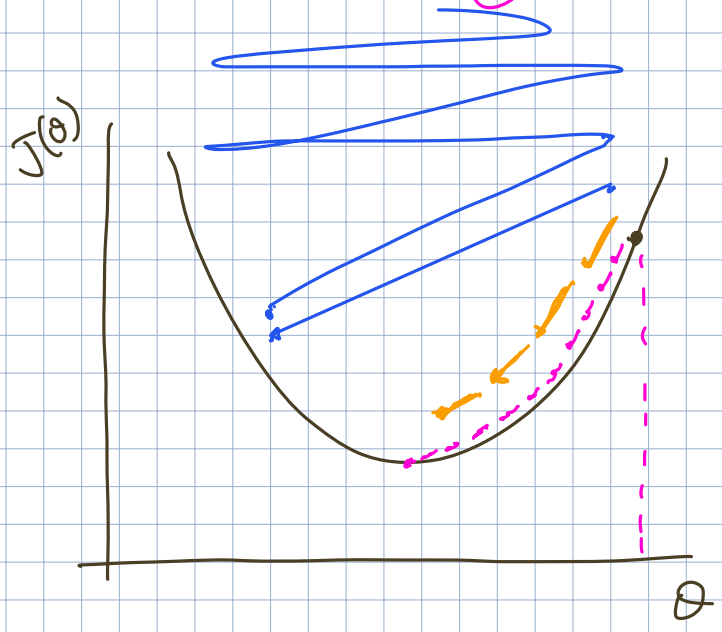
$$f^{[1]}(x_1) + f^{[2]}(x_1) = 0.8 + 0.1 = 0.9 = y_1$$
$$f^{[1]}(x_2) + f^{[2]}(x_2) = 1.5 - 0.1 = 1.4 = y_2$$
$$f^{[1]}(x_3) + f^{[2]}(x_3) = 6.0 + 0.2 = 6.2 = y_3$$

Algoritmos de gradient descent:



learning rate

$$\theta_{i+1} = \theta_i - \rho \frac{\partial J(\theta)}{\partial \theta}$$



- learning rate pequeño
- LR mediano
- LR alto

cómo se conecta esto con XGboost?

$$J(y, f(x)) = \sum_{i=1}^{N} \frac{(f(x_i) - y_i)^2}{2}$$

$$\frac{\partial J(y_i, f(x_i))}{\partial f(x_i)} = f(x_i) - y_i$$ → error del modelo.

En XGboost: $f^{[m+1]}(x_i) = f^{[m]}(x_i) - \rho\, g^{[m]}(x_i)$

↳ modelo que estima los errores:

$$\frac{\partial J(y_i, f^{[m]}(x_i))}{\partial f(x_i)} = f^{[m]}(x_i) - y_i$$